

Topic:

Network Security

Subject: Computer Science

Research Question:

“How Vulnerable is a Home Router? A
Penetration Testing Study Using Wireshark”

Word Count: 3867

Table of Contents

Research Question:.....	1
“How Vulnerable is a Home Router? A Penetration Testing Study Using Wireshark”.....	1
Table of Contents.....	2
Introduction.....	3
Background.....	3
Network Protocols.....	4
How home Networks work.....	4
Core Components.....	5
Network Types.....	5
Wireshark.....	5
Detailed Features and Functions of Wireshark.....	5
Methodology.....	6
Practical Work.....	7
Wireshark.....	8
HTTP Login Page Test.....	12
DNS Lookup Test.....	16
ARP Test.....	18
Analysis.....	21
Conclusion.....	22
Evaluation & Limitations.....	23
Bibliography.....	24

Introduction

In today's hyperconnected digital world, networks have become a foundational component of our lives. We can work remotely, study online, and manage our finances. As network usage grows rapidly, the risk of personal information theft increases accordingly. 1.1 million people in the United States had their personal data stolen, and the number of individuals affected is growing (Experian, 2024). This Extended Essay investigates one of the most relevant topics in modern cybersecurity and aims to answer the following question: "How Vulnerable is a Home Router?" This question is under the topics of Computer Science as it is focused on the decisions regarding router's firmware that are creating weaknesses in the network. This investigation is not focused on user behaviour or general cybercrime, it is focused on weaknesses and solutions of a router with a default settings.

Understanding these concepts provides a necessary foundation for the following background section, which will explore key principles relevant to assessing home router vulnerability.



(Figure 1.1 - Wireshark)

Using **Wireshark** (figure 1.1), a widely used open-source packet analyzer, this investigation simulates a basic penetration test to provide insight into what kinds of data might be exposed to attackers on an unsecured network.

This study aims not only to uncover potential weaknesses in a common home setup but also to suggest basic, practical security practices that everyday users can implement, such as updating firmware or enabling security settings not offered by default by the internet provider.

Background

Network Protocols

The most used internet protocols are

Protocol	Full Name	Purpose	Typical Port(s)
TCP/IP	Transmission Control Protocol / Internet Protocol	Core protocol suite for internet communication.	—
UDP	User Datagram Protocol	Used in DNS communications and fast, connectionless transfers.	—
HTTP	Hypertext Transfer Protocol	Used in the login test; shows vulnerability due to lack of encryption.	80
HTTPS	Hypertext Transfer Protocol Secure	Mentioned as the secure alternative to HTTP.	443
DNS	Domain Name System	Used to resolve website names (e.g., youtube.com) to IP addresses.	53 (TCP/UDP)
ARP	Address Resolution Protocol	Used in ARP test to map IP addresses to MAC addresses.	No port (data link layer)

(Table 1.1 - Network protocols with their explanations)

(Auvik Networks, 2025; TechTarget, 2025; Iana, 2025).

How home Networks work

A home network connects devices like computers, smartphones, tablets, smart TVs, printers, and game consoles within a residence, enabling communication and resource sharing, including internet access and file sharing.

Core Components

- Access Point: Manages wireless connections. They are usually integrated into **routers**.
- Router: The central network hub, managing both wired (Ethernet) and wireless (Wi-Fi) connections. Modern routers often integrate wireless access point and **switch** functions, connecting devices internally and to the internet.
- Switch: An Optional device used to increase the number of wired LAN ports.
- Modem: Connects the home network to the Internet Service Provider (ISP).
Modern setups often combine a modem and a router into a single device.

Network Types

Type	Full Name	Explanation
LAN	Local Area Network	Connects local devices in a small area.
WLAN	Wireless LAN	Wi-Fi network in a home or office.
VPN	Virtual Private Network	Encrypted connection over public network.
WAN	Wide Area Network	Connects home network to the Internet.

(Table 1.2 - Network Types)

Wireshark

Wireshark is a free and widely used network protocol analyzer that captures and analyzes network traffic (Wireshark Foundation, 2025). It allows users to view network packets as they traverse a network, showing details such as source and destination addresses, protocols, and packet contents(Wireshark Foundation, 2025). Professionals

and security researchers use Wireshark to troubleshoot network issues(Wireshark Foundation, 2025).

Detailed Features and Functions of Wireshark

Feature / Use	Description
Packet Capture	Captured live traffic.
Protocol Support	Used TCP, UDP, HTTP, DNS, ARP.
Filtering	Isolated HTTP, DNS, ARP traffic.
Packet Details	Viewed headers and payloads.
Reconnaissance	Found IP/MAC via ARP, DNS.
Unencrypted Data	Exposed login via HTTP.
Credential Analysis	Extracted HTTP form data.
Interface Selection	Picked the correct Wi-Fi adapter.
Capture Filters	Applied DNS/HTTP/ARP filters.
Live Capture & Stop	Captured and stopped tests.
Analysis	Inspected packet content.
Encrypted Traffic	Not readable without keys.

Legal Use	Done on own network.
-----------	----------------------

(Table 1.3 - Wireshark functions)

Methodology

This study has used a structured penetration testing style guidance regarding passive packet capture. All the tests shown in this investigation were done only on the researcher's home network using only the researcher's routers with default configurations. This decision was made to practically emphasize the real world examples of non-experts operating their devices. During this research the only tool for testing was Wireshark. It was selected for this study because it has the ability to inspect packets on different levels of OSI model, filter specific data types such as HTTP, DNS and ARP, and rebuild payloads without distracting the live traffic of the network. The test was done using three user devices that were connected to a single router with model G-97RGT. This investigation focused only on passive observations to ensure that any discovered weaknesses operated normally and not after any manipulations.

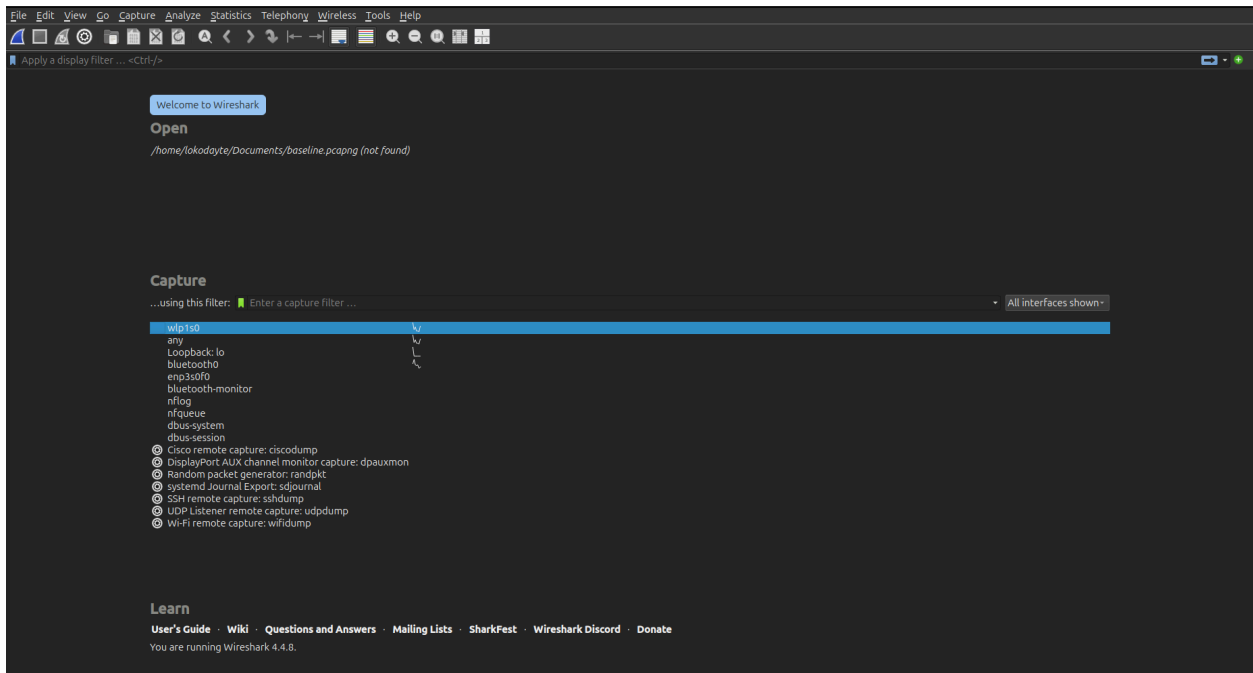
Three tests were chosen: an HTTP login test, a DNS lookup test, and an ARP observation. These represent realistic threats at the application, transport, and data link layers. The tests show how an unauthorized user without special access could exploit certain vulnerabilities. No active attacks, such as ARP spoofing or packet injection, were performed, and no custom firmware was used. This approach kept the focus on default router behavior and followed ethical guidelines.

Other tools like tcpdump and Nmap were also considered, but Wireshark was chosen because it can examine network data more closely and presents information in a way that is easier to understand. Each test was designed to answer the main research question, demonstrating that a typical home router cannot detect or stop these quiet threats. This enabled the identification of real security problems in regular home networks.

Practical Work

In this section the practical part of the investigation is presented, while using example testing machines that are publicly accessible. The analysis part is based on this practical component

Wireshark



(Figure 1.2 - Wireshark bootup screen)

When we start Wireshark, we must select the correct network interface for packet capture. For internet testing, select the interface responsible for packet transmission. In Figure 1.2, it is wlp1s0. To identify the relevant network interface in Ubuntu, run the terminal command 'iwconfig' to display interface details as in Figure 1.3.

```

lokodayte@lokodayte:~$ iwconfig
lo                no wireless extensions.

enp3s0f0          no wireless extensions.

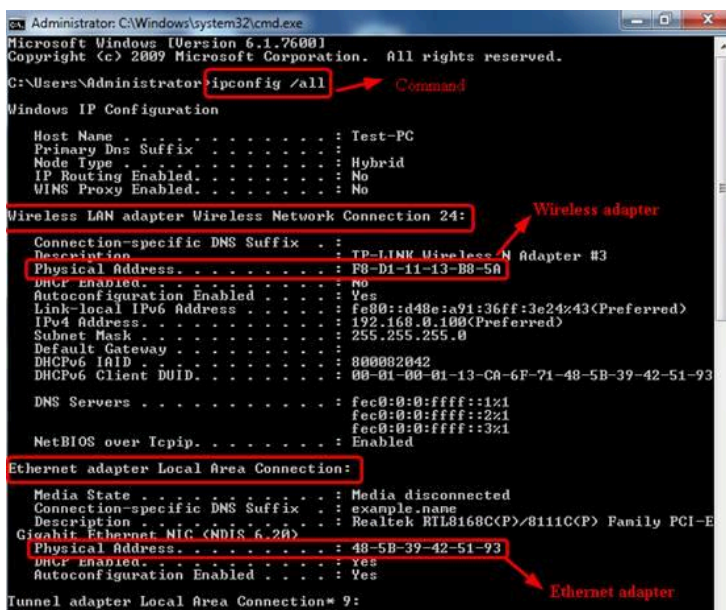
wlp1s0            IEEE 802.11  ESSID:"UMCDillijan-College"
Mode:Managed    Frequency:5.58 GHz  Access Point: 6C:C4:9F:EF:FF:50
Tx-Power=22 dBm
  Retry short limit:7   RTS thr:off   Fragment thr:off
Power Management:on
Link Quality=48/70  Signal level=-62 dBm
Rx invalid nwid:0    Rx invalid crypt:0 Rx invalid frag:0
Tx excessive retries:1 Invalid misc:16  Missed beacon:0

lokodayte@lokodayte:~$

```

(Figure 1.3 - Ubuntu terminal)

To view the network interface in Windows, open the Command Prompt and run the command. **ipconfig /all**, there we will see information about interfaces. The screen that will appear should look like Figure 1.4



```

Administrator: C:\Windows\system32\cmd.exe
Microsoft Windows [Version 6.1.7600]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\Administrator>ipconfig /all
Windows IP Configuration

Host Name . . . . . : Test-PC
Primary Dns Suffix . . . . . :
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No

Wireless LAN adapter Wireless Network Connection 24:
Connection-specific DNS Suffix . :
Description . . . . . : TP-LINK Wireless N Adapter #3
Physical Address. . . . . : 98-D1-11-13-B8-5A
Dhcp Enabled. . . . . : No
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80::d48e:a91:36ff:3e24%43(Preferred)
IPv4 Address. . . . . : 192.168.0.100(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . :
Dhcpv6 IAID . . . . . : 800082042
Dhcpv6 Client DUID. . . . . : 00-01-00-01-13-C8-6F-71-48-5B-39-42-51-93

DNS Servers . . . . . : fec0:0:0:ffff::1%1
                       fec0:0:0:ffff::2%1
                       fec0:0:0:ffff::3%1
NetBIOS over Tcpip. . . . . : Enabled

Ethernet adapter Local Area Connection:
Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . : example.name
Description . . . . . : Realtek RTL8168C(P)/8111C(P) Family PCI-E
Gigabit Ethernet NIC (NDIS 6.20)
Physical Address. . . . . : 48-5B-39-42-51-93
Dhcp Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes

Tunnel adapter Local Area Connection* 9:

```

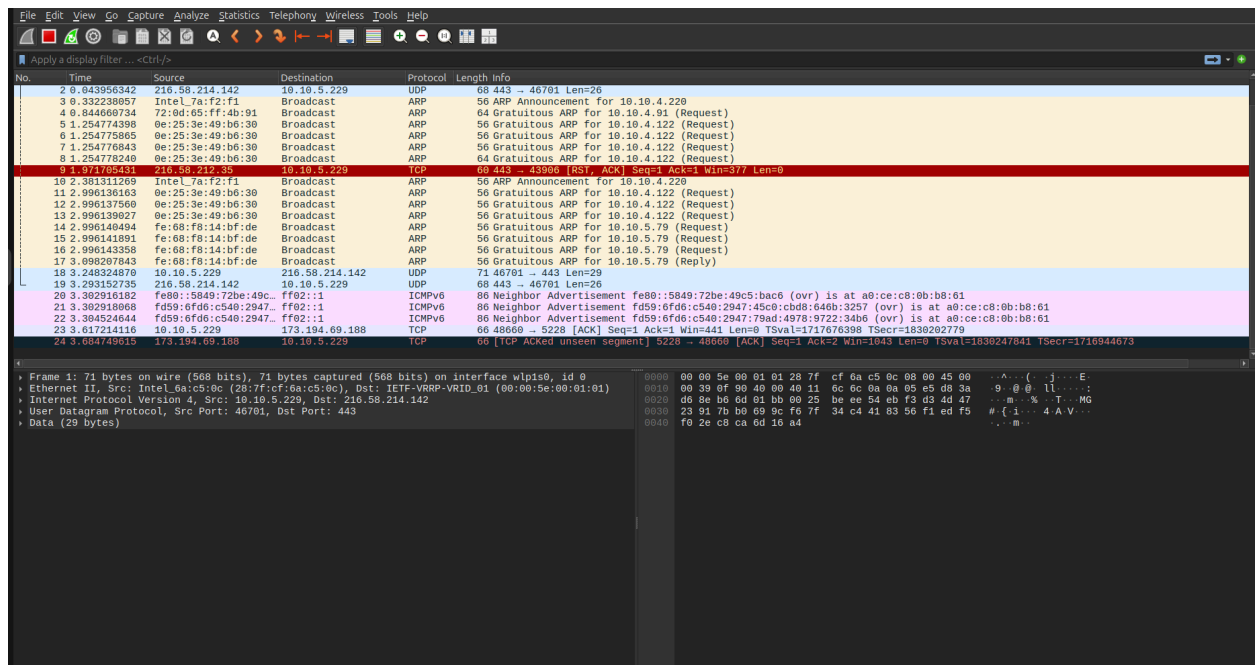
Annotations in the image:

- A red box around the command `ipconfig /all` is labeled "Command".
- A red box around the section header "Wireless LAN adapter Wireless Network Connection 24:" is labeled "Wireless adapter".
- A red box around the section header "Ethernet adapter Local Area Connection:" is labeled "Ethernet adapter".

(Figure 1.4 - Windows network interface in CMD)

The line above the Wireless **adapter** shows the name of our interface.

When the right interface is chosen, the screen should appear as in **Figure 1.5**.



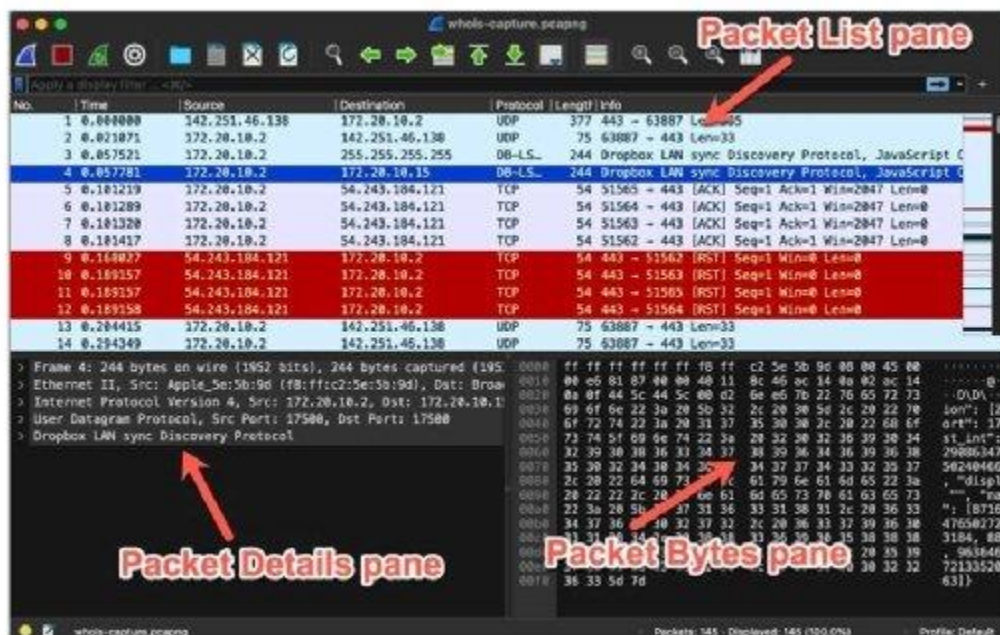
(Figure 1.5 - Wireshark screen for packet capturing)

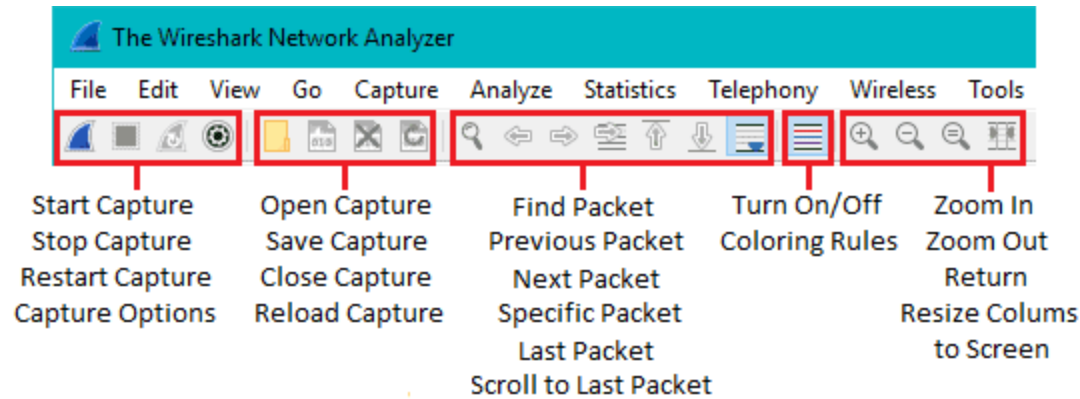
In the screen shown in **Figure 1.5**, we see network traffic. Here is information about the colors of the packets (each packet represents a formatted unit of data, typically a fragment of a larger message).

- Light purple for TCP traffic
- Light blue for UDP traffic
- Black for packets with errors

- Light green for HTTP traffic
- Light yellow for Windows-specific traffic, such as SMB and NetBIOS
- Dark yellow for routing protocols
- Dark gray for TCP control packets like SYN, FIN, and ACK.(Wireshark Foundation, 2025)

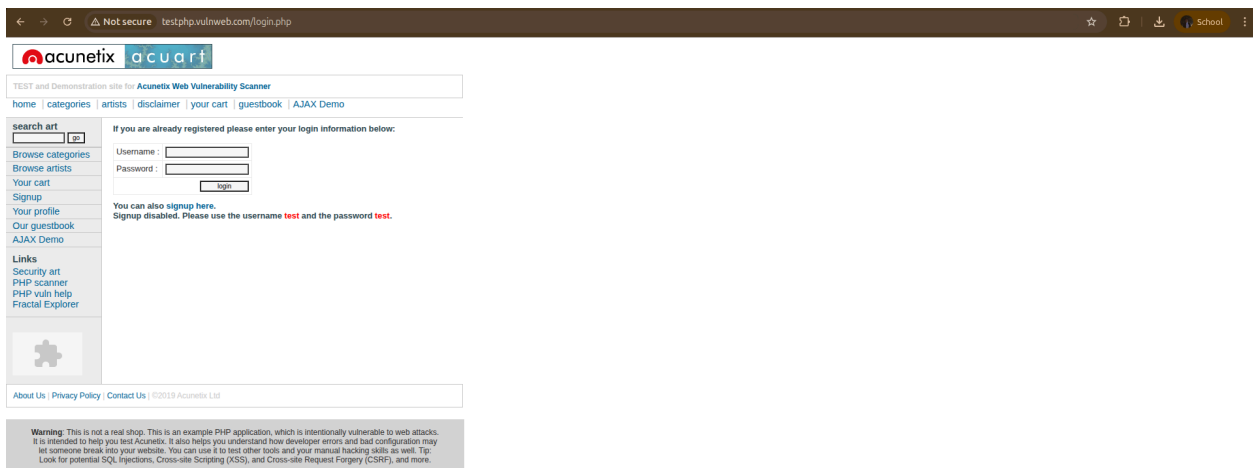
Figure 1.6 provides detailed information about **Wireshark** configurations.





(Figure 1.6 - Wireshark user instructions)

HTTP Login Page Test



(Figure 2.1 - HTTP website for tests)

The website shown in Figure 2.1 is for testing. To start with practical work, we open Wireshark and start it.

Fill in the login page and press the login button. As shown in Figure 2.2



TEST and Demonstration site for [Acunetix Web Vulnerability Scanner](#)

[home](#) | [categories](#) | [artists](#) | [disclaimer](#) | [your cart](#) | [guestbook](#) | [AJAX Demo](#)

search art

GO

[Browse categories](#)

[Browse artists](#)

[Your cart](#)

[Signup](#)

[Your profile](#)

[Our guestbook](#)

[AJAX Demo](#)

Links

[Security art](#)

[PHP scanner](#)

[PHP vuln help](#)

[Fractal Explorer](#)



If you are already registered please enter your login information below:

Username :

Password :

You can also [signup here](#).

Signup disabled. Please use the username **test** and the password **test**.

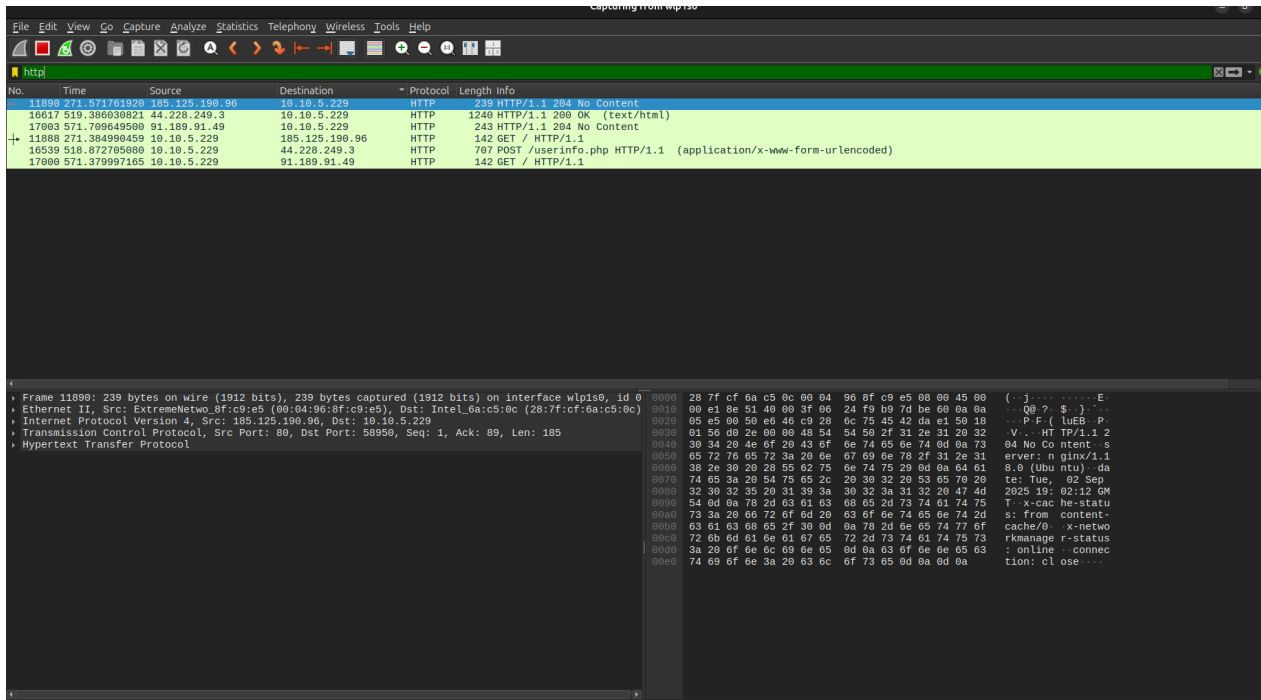
[About Us](#) | [Privacy Policy](#) | [Contact Us](#) | ©2019 Acunetix Ltd

Warning: This is not a real shop. This is an example PHP application, which is intentionally vulnerable to web attacks. It is intended to help you test Acunetix. It also helps you understand how developer errors and bad configuration may let someone break into your website. You can use it to test other tools and your manual hacking skills as well. Tip: Look for potential SQL injections, Cross-site Scripting (XSS), and Cross-site Request Forgery (CSRF), and more.

(Figure 2.2 - HTTP login test page with written credentials)

As an example, in both fields, the word 'test' was entered.

Since the website uses **HTTP**, we need to filter for **HTTP** in **Wireshark**.



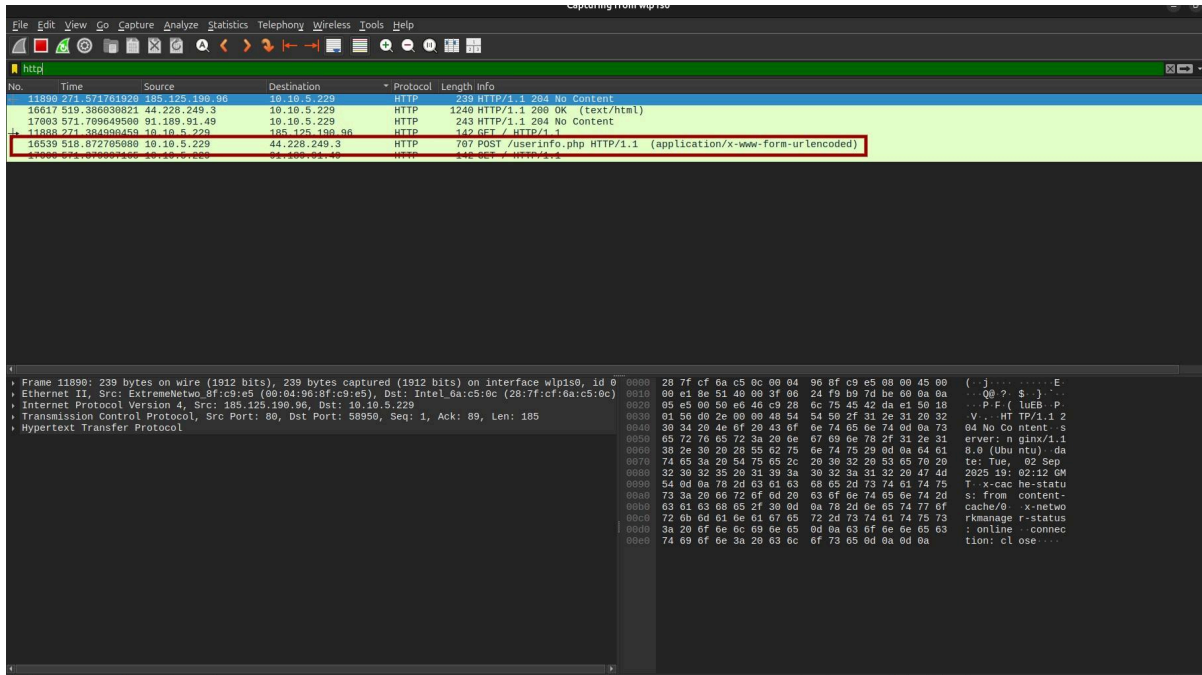
(Figure 2.3 - Filtered Wireshark packets)

Currently, we have 6 packets, each containing information about the HTTP destinations the laptop accessed. In one of them, there is information about the login page where the test was done.

In this case, the information that we need is located in this packet. We can tell the key indicators that this packet likely contains information about the login page are:

- The HTTP method is POST, commonly used for login forms to submit user credentials.
- The URL path "/userinfo.php" suggests it is related to user information or authentication.

- The content type "application/x-www-form-urlencoded" is the standard encoding for form submissions.



(Figure 2.4 - Wireshark packet selected)

- **Frame 16539: 707 bytes on wire (5656 bits), 707 bytes captured (5656 bits) on interface wlp1s0**

This is the packet number (16539), the total size of the packet in bytes and bits as it was transmitted on the network, and the number of bytes Wireshark captured for analysis. The interface name (wlp1s0) is the network device where the packet was captured.

- **Ethernet II, Src: Intel_6a:c5:0c (28:7f:cf:6a:c5:0c), Dst: IETF-VRRP-VRID_01 (00:00:5e:00:01:01)**

This is the data link layer header. It shows the source MAC address (Intel_6a:c5:0c), destination MAC address (IETF-VRRP-VRID_01), and the Ethernet protocol type. This layer handles communication on a local network segment.

- **Internet Protocol Version 4, Src: 10.10.5.229, Dst: 44.228.249.3**

This is the network layer header providing IPv4 addressing information, showing the source IP address (10.10.5.229) and destination IP address (44.228.249.3). It routes the packet across multiple networks to reach the destination.

- **Transmission Control Protocol, Src Port: 38538, Dst Port: 80, Seq: 1, Ack: 1, Len: 653**

This is the TCP transport-layer header. It includes the source port (38538) and destination port (80, the standard for HTTP), the sequence and acknowledgment numbers for reliable data transfer, and the length of the TCP payload (653 bytes). TCP ensures ordered and reliable communication.

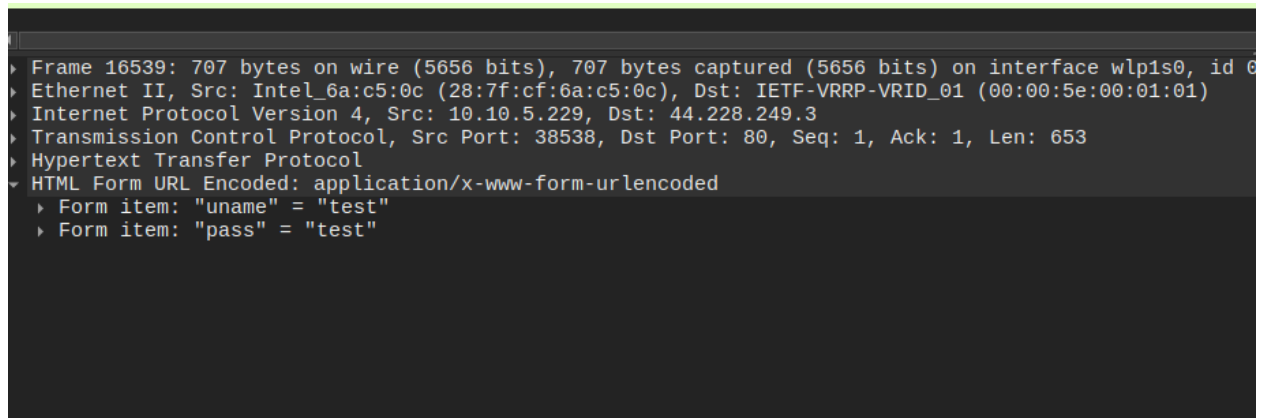
- **Hypertext Transfer Protocol**

This is the application layer showing the HTTP protocol data. It includes the POST method, indicating the submission of form data, the target resource (/userinfo.php), and the HTTP version 1.1.

- **HTML Form URL Encoded: application/x-www-form-urlencoded**

This specifies that the HTTP POST data is encoded as form data using key-value pairs, as in typical web form submissions. This encoding type sends form inputs as URL-encoded strings in the HTTP body.

If we open the last line **HTML Form...**, we will have access to the information that we were not supposed to have access to



```
Frame 16539: 707 bytes on wire (5656 bits), 707 bytes captured (5656 bits) on interface wlp1s0, id 6
Ethernet II, Src: Intel_6a:c5:0c (28:7f:cf:6a:c5:0c), Dst: IETF-VRRP-VRID_01 (00:00:5e:00:01:01)
Internet Protocol Version 4, Src: 10.10.5.229, Dst: 44.228.249.3
Transmission Control Protocol, Src Port: 38538, Dst Port: 80, Seq: 1, Ack: 1, Len: 653
Hypertext Transfer Protocol
HTML Form URL Encoded: application/x-www-form-urlencoded
  Form item: "uname" = "test"
  Form item: "pass" = "test"
```

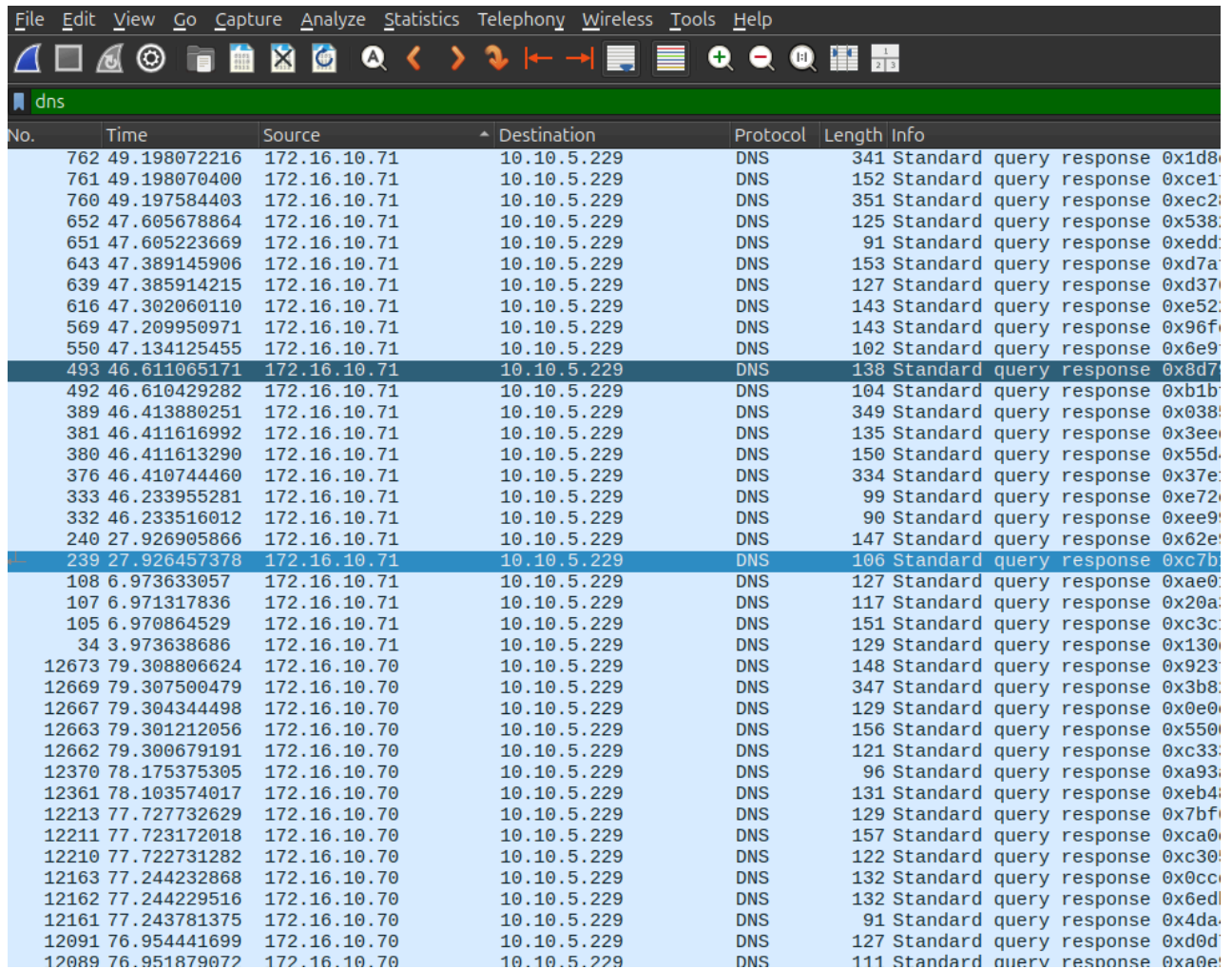
(Figure 2.5 - Information provided in Wireshark)

This test emphasizes that sensitive information, such as login credentials, can be transmitted in plain text over HTTP connections, which is insecure. The ability to capture data that should stay private highlights a significant vulnerability in home networks. It shows that without encryption, users are at risk of credential theft and unauthorized access.

DNS Lookup Test

To start with this test, we are starting **Wireshark** the same way. While it captures data in the browser. While Wireshark captured data, a few websites were opened (bbc.com, wikipedia.org, youtube.com) to continue the investigation.

Now in **Wireshark**, we filter the traffic and set **DNS** to see the traffic we need this time.



No.	Time	Source	Destination	Protocol	Length	Info
762	49.198072216	172.16.10.71	10.10.5.229	DNS	341	Standard query response 0x1d8
761	49.198070400	172.16.10.71	10.10.5.229	DNS	152	Standard query response 0xce1
760	49.197584403	172.16.10.71	10.10.5.229	DNS	351	Standard query response 0xec2
652	47.605678864	172.16.10.71	10.10.5.229	DNS	125	Standard query response 0x538
651	47.605223669	172.16.10.71	10.10.5.229	DNS	91	Standard query response 0xedd
643	47.389145906	172.16.10.71	10.10.5.229	DNS	153	Standard query response 0xd7a
639	47.385914215	172.16.10.71	10.10.5.229	DNS	127	Standard query response 0xd37
616	47.302060110	172.16.10.71	10.10.5.229	DNS	143	Standard query response 0xe52
569	47.209950971	172.16.10.71	10.10.5.229	DNS	143	Standard query response 0x96f
550	47.134125455	172.16.10.71	10.10.5.229	DNS	102	Standard query response 0x6e9
493	46.611065171	172.16.10.71	10.10.5.229	DNS	138	Standard query response 0x8d7
492	46.610429282	172.16.10.71	10.10.5.229	DNS	104	Standard query response 0xb1b
389	46.413880251	172.16.10.71	10.10.5.229	DNS	349	Standard query response 0x038
381	46.411616992	172.16.10.71	10.10.5.229	DNS	135	Standard query response 0x3ee
380	46.411613290	172.16.10.71	10.10.5.229	DNS	150	Standard query response 0x55d
376	46.410744460	172.16.10.71	10.10.5.229	DNS	334	Standard query response 0x37e
333	46.233955281	172.16.10.71	10.10.5.229	DNS	99	Standard query response 0xe72
332	46.233516012	172.16.10.71	10.10.5.229	DNS	90	Standard query response 0xee9
240	27.926905866	172.16.10.71	10.10.5.229	DNS	147	Standard query response 0x62e
239	27.926457378	172.16.10.71	10.10.5.229	DNS	106	Standard query response 0xc7b
108	6.973633057	172.16.10.71	10.10.5.229	DNS	127	Standard query response 0xae0
107	6.971317836	172.16.10.71	10.10.5.229	DNS	117	Standard query response 0x20a
105	6.970864529	172.16.10.71	10.10.5.229	DNS	151	Standard query response 0xc3c
34	3.973638686	172.16.10.71	10.10.5.229	DNS	129	Standard query response 0x130
12673	79.308806624	172.16.10.70	10.10.5.229	DNS	148	Standard query response 0x923
12669	79.307500479	172.16.10.70	10.10.5.229	DNS	347	Standard query response 0x3b8
12667	79.304344498	172.16.10.70	10.10.5.229	DNS	129	Standard query response 0x0e0
12663	79.301212056	172.16.10.70	10.10.5.229	DNS	156	Standard query response 0x550
12662	79.300679191	172.16.10.70	10.10.5.229	DNS	121	Standard query response 0xc33
12370	78.175375305	172.16.10.70	10.10.5.229	DNS	96	Standard query response 0xa93
12361	78.103574017	172.16.10.70	10.10.5.229	DNS	131	Standard query response 0xeb4
12213	77.727732629	172.16.10.70	10.10.5.229	DNS	129	Standard query response 0x7bf
12211	77.723172018	172.16.10.70	10.10.5.229	DNS	157	Standard query response 0xca0
12210	77.722731282	172.16.10.70	10.10.5.229	DNS	122	Standard query response 0xc30
12163	77.244232868	172.16.10.70	10.10.5.229	DNS	132	Standard query response 0x0cc
12162	77.244229516	172.16.10.70	10.10.5.229	DNS	132	Standard query response 0x6ed
12161	77.243781375	172.16.10.70	10.10.5.229	DNS	91	Standard query response 0x4da
12091	76.954441699	172.16.10.70	10.10.5.229	DNS	127	Standard query response 0xd0d
12089	76.951879072	172.16.10.70	10.10.5.229	DNS	111	Standard query response 0xa0e

(Figure 3.1 - Wireshark filter for DNS packets)

In a few seconds, **Wireshark** captured 14887 packets, which are impossible to analyze manually. I used a bit more advanced filtering. I filtered only the websites I wanted to test. The filter looks like this: "dns.qry.name contains "bbc.com" or dns.qry.name contains "youtube.com" or dns.qry.name contains "wikipedia.org".

189	50.289357806	172.16.10.71	10.10.5.229	DNS	205	Standard	query	response	0x8f00	A gn-w
177	50.276034098	172.16.10.71	10.10.5.229	DNS	141	Standard	query	response	0x6243	HTTPS
115	49.783045421	172.16.10.71	10.10.5.229	DNS	149	Standard	query	response	0x7155	A www.
113	49.781905933	172.16.10.71	10.10.5.229	DNS	133	Standard	query	response	0x60bc	HTTPS
150	49.528994485	172.16.10.71	10.10.5.229	DNS	126	Standard	query	response	0x3b38	HTTPS
199	49.279419441	172.16.10.71	10.10.5.229	DNS	126	Standard	query	response	0x7aa2	HTTPS
197	49.275901865	172.16.10.71	10.10.5.229	DNS	131	Standard	query	response	0x6703	A bbc.
179	49.265970288	172.16.10.71	10.10.5.229	DNS	126	Standard	query	response	0xedd6	HTTPS
163	79.301212056	172.16.10.70	10.10.5.229	DNS	156	Standard	query	response	0x5500	HTTPS
162	79.300679191	172.16.10.70	10.10.5.229	DNS	121	Standard	query	response	0xc333	A en.w
111	77.723172018	172.16.10.70	10.10.5.229	DNS	157	Standard	query	response	0xca0c	HTTPS
110	77.722731282	172.16.10.70	10.10.5.229	DNS	122	Standard	query	response	0xc305	A www.
185	67.135909407	172.16.10.70	10.10.5.229	DNS	108	Standard	query	response	0xd7eb	HTTPS
184	67.135549064	172.16.10.70	10.10.5.229	DNS	124	Standard	query	response	0xf7c1	A acco
184	66.053888990	172.16.10.70	10.10.5.229	DNS	86	Standard	query	response	0xbddf	HTTPS
183	66.053440361	172.16.10.70	10.10.5.229	DNS	87	Standard	query	response	0x02bc	A yout
162	62.736588689	172.16.10.70	10.10.5.229	DNS	124	Standard	query	response	0xb7ca	HTTPS
161	62.736080550	172.16.10.70	10.10.5.229	DNS	365	Standard	query	response	0x4be0	A www.
161	79.295703478	10.10.5.229	172.16.10.70	DNS	76	Standard	query	0x5500	HTTPS	en.wikip
160	79.295472982	10.10.5.229	172.16.10.70	DNS	76	Standard	query	0xc333	A en.wikipedia.	
109	77.719075753	10.10.5.229	172.16.10.70	DNS	77	Standard	query	0xca0c	HTTPS	www.wikip
108	77.718900087	10.10.5.229	172.16.10.70	DNS	77	Standard	query	0xc305	A www.wikipedia	
176	67.131384350	10.10.5.229	172.16.10.70	DNS	80	Standard	query	0xd7eb	HTTPS	accounts.
174	67.131048874	10.10.5.229	172.16.10.70	DNS	80	Standard	query	0xf7c1	A accounts.yout	
144	65.977321914	10.10.5.229	172.16.10.70	DNS	71	Standard	query	0xbddf	HTTPS	youtube.c
143	65.977159939	10.10.5.229	172.16.10.70	DNS	71	Standard	query	0x02bc	A youtube.com	
160	62.732970949	10.10.5.229	172.16.10.70	DNS	75	Standard	query	0xb7ca	HTTPS	www.youtu
159	62.732602645	10.10.5.229	172.16.10.70	DNS	75	Standard	query	0x4be0	A www.youtube.c	
122	50.148269471	10.10.5.229	172.16.10.71	DNS	85	Standard	query	0x6243	HTTPS	gn-web-as
121	50.148105121	10.10.5.229	172.16.10.71	DNS	85	Standard	query	0x8f00	A gn-web-assets	
189	49.676911784	10.10.5.229	172.16.10.71	DNS	71	Standard	query	0x60bc	HTTPS	www.bbc.c
188	49.676605015	10.10.5.229	172.16.10.71	DNS	71	Standard	query	0x7155	A www.bbc.com	
149	49.526924144	10.10.5.229	172.16.10.71	DNS	67	Standard	query	0x3b38	HTTPS	bbc.com
198	49.277432078	10.10.5.229	172.16.10.71	DNS	67	Standard	query	0x7aa2	HTTPS	bbc.com
165	49.200347908	10.10.5.229	172.16.10.71	DNS	67	Standard	query	0xedd6	HTTPS	bbc.com
164	49.200044212	10.10.5.229	172.16.10.71	DNS	67	Standard	query	0x6703	A bbc.com	

(Figure 3.2 - Filter only webpage names)

After filtering we got only 36 packets which is now possible to even analyze manually.

172.16.10.71	10.10.5.229	DNS	205 Standard query response 0x8f00 A gn-web-assets.api.bbc.com CNAME static-web-assets.gnl-common.bbcverticals.
172.16.10.71	10.10.5.229	DNS	141 Standard query response 0x6243 HTTPS gn-web-assets.api.bbc.com CNAME static-web-assets.gnl-common.bbcverticals.
172.16.10.71	10.10.5.229	DNS	149 Standard query response 0x7155 A www.bbc.com CNAME www.bbc.com.pri.bbc.com CNAME bbc.map.fastly.net A 146.71.146.71
172.16.10.71	10.10.5.229	DNS	133 Standard query response 0x60bc HTTPS www.bbc.com CNAME www.bbc.com.pri.bbc.com CNAME bbc.map.fastly.net
172.16.10.71	10.10.5.229	DNS	126 Standard query response 0x3b38 HTTPS bbc.com SOA ns.bbc.co.uk
172.16.10.71	10.10.5.229	DNS	126 Standard query response 0x7aa2 HTTPS bbc.com SOA ns.bbc.co.uk
172.16.10.71	10.10.5.229	DNS	131 Standard query response 0x6703 A bbc.com A 151.101.0.81 A 151.101.64.81 A 151.101.128.81 A 151.101.192.81
172.16.10.71	10.10.5.229	DNS	126 Standard query response 0xadd6 HTTPS bbc.com SOA ns.bbc.co.uk
172.16.10.70	10.10.5.229	DNS	156 Standard query response 0x5500 HTTPS en.wikipedia.org CNAME dyna.wikimedia.org SOA ns0.wikimedia.org
172.16.10.70	10.10.5.229	DNS	121 Standard query response 0xc333 A en.wikipedia.org CNAME dyna.wikimedia.org A 185.15.59.224
172.16.10.70	10.10.5.229	DNS	157 Standard query response 0xca9c HTTPS www.wikipedia.org CNAME dyna.wikimedia.org SOA ns0.wikimedia.org
172.16.10.70	10.10.5.229	DNS	122 Standard query response 0xc305 A www.wikipedia.org CNAME dyna.wikimedia.org A 185.15.59.224
172.16.10.70	10.10.5.229	DNS	108 Standard query response 0xd7eb HTTPS accounts.youtube.com CNAME www3.l.google.com
172.16.10.70	10.10.5.229	DNS	124 Standard query response 0xf7c1 A accounts.youtube.com CNAME www3.l.google.com A 216.58.212.46
172.16.10.70	10.10.5.229	DNS	86 Standard query response 0xbdbf HTTPS youtube.com HTTPS
172.16.10.70	10.10.5.229	DNS	87 Standard query response 0x02bc A youtube.com A 142.251.140.14
172.16.10.70	10.10.5.229	DNS	124 Standard query response 0xb7ca HTTPS www.youtube.com CNAME youtube-ui.l.google.com HTTPS
172.16.10.70	10.10.5.229	DNS	365 Standard query response 0x4be0 A www.youtube.com CNAME youtube-ui.l.google.com A 216.58.214.142 A 142.250.142.142
10.10.5.229	172.16.10.70	DNS	76 Standard query 0x5500 HTTPS en.wikipedia.org
10.10.5.229	172.16.10.70	DNS	76 Standard query 0xc333 A en.wikipedia.org
10.10.5.229	172.16.10.70	DNS	77 Standard query 0xca0c HTTPS www.wikipedia.org
10.10.5.229	172.16.10.70	DNS	77 Standard query 0xc305 A www.wikipedia.org
10.10.5.229	172.16.10.70	DNS	80 Standard query 0xd7eb HTTPS accounts.youtube.com
10.10.5.229	172.16.10.70	DNS	80 Standard query 0xf7c1 A accounts.youtube.com
10.10.5.229	172.16.10.70	DNS	71 Standard query 0xbdbf HTTPS youtube.com
10.10.5.229	172.16.10.70	DNS	71 Standard query 0x02bc A youtube.com
10.10.5.229	172.16.10.70	DNS	75 Standard query 0xb7ca HTTPS www.youtube.com
10.10.5.229	172.16.10.70	DNS	75 Standard query 0x4be0 A www.youtube.com
10.10.5.229	172.16.10.71	DNS	85 Standard query 0x6243 HTTPS gn-web-assets.api.bbc.com
10.10.5.229	172.16.10.71	DNS	85 Standard query 0x8f00 A gn-web-assets.api.bbc.com
10.10.5.229	172.16.10.71	DNS	71 Standard query 0x60bc HTTPS www.bbc.com
10.10.5.229	172.16.10.71	DNS	71 Standard query 0x7155 A www.bbc.com
10.10.5.229	172.16.10.71	DNS	67 Standard query 0x3b38 HTTPS bbc.com
10.10.5.229	172.16.10.71	DNS	67 Standard query 0x7aa2 HTTPS bbc.com
10.10.5.229	172.16.10.71	DNS	67 Standard query 0xedd6 HTTPS bbc.com
10.10.5.229	172.16.10.71	DNS	67 Standard query 0x6703 A bbc.com

(Figure 3.3 - DNS query and response traffic)

The screenshot above shows **DNS query and response traffic** captured in Wireshark when accessing *bbc.com*, *wikipedia.org*, and *youtube.com*. Each line represents either a **DNS query** (a request for an IP address) or a **DNS response** (the answer from the DNS server with the corresponding IP).

- For **BBC**:
- We see queries such as **www.bbc.com** and **CNAME** records pointing to **bbc.map.fastly.net**. The response then provides multiple IP addresses (e.g., **151.101.0.81**, **151.101.64.81**). This emphasizes that, even though the BBC uses a content delivery network (Fastly), DNS traffic clearly shows that the user tried to visit the BBC's website.
-
- For **Wikipedia**:

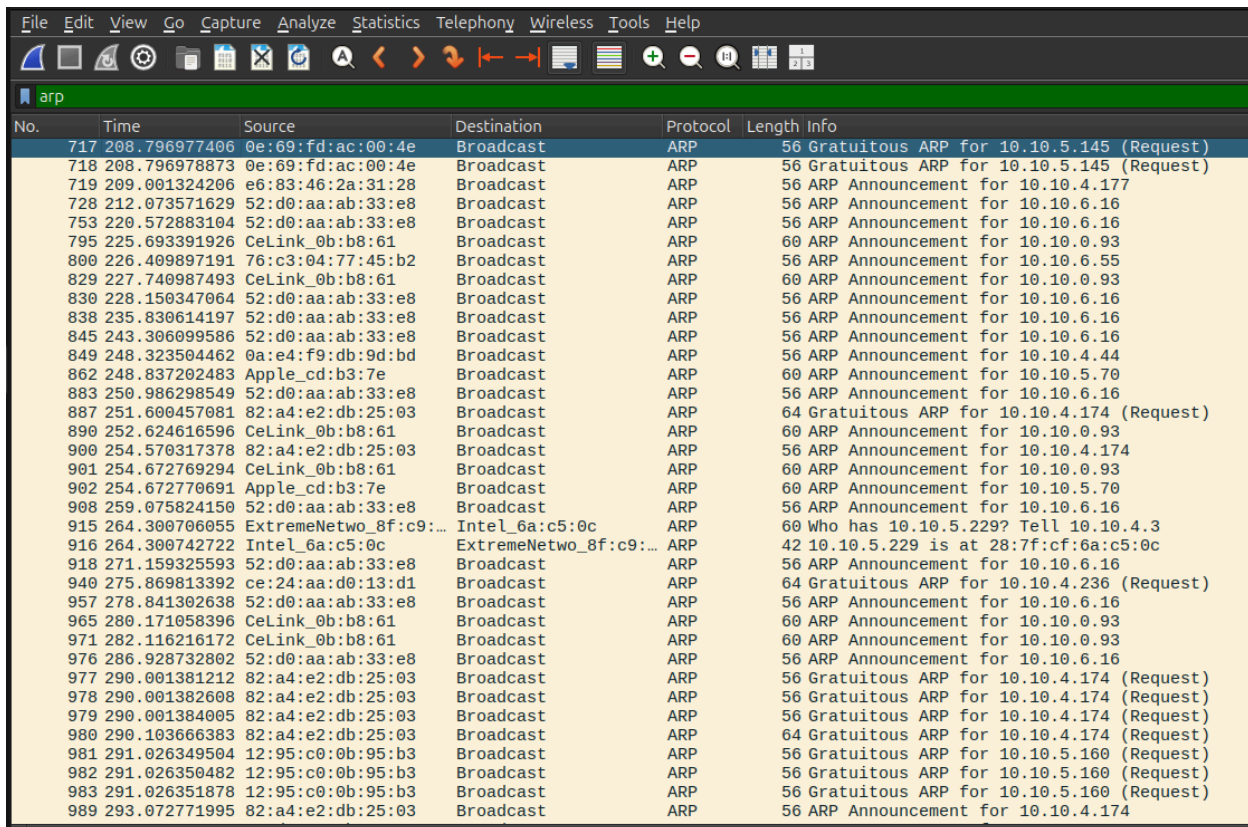
- Packets such as **A en.wikipedia.org** return IP addresses like **185.15.59.224**. We also see things on Wikimedia.org, the hosting organization for Wikipedia. Again, this shows the exact site being accessed.
-
- For **YouTube**:
- Multiple DNS queries resolve domains such as **www.youtube.com**, **accounts.youtube.com**, and **youtube-ui.l.google.com**. These point to IP addresses owned by Google (e.g., **216.58.212.46**, **142.250.140.14**). This again shows that a single visit to **YouTube** often results in multiple DNS lookups for related subdomains.

Although the actual browsing sessions are protected by **HTTPS encryption**, the DNS queries themselves are unencrypted (in this test setup)(Herzberg, 2020). This means that anyone with access to network traffic, such as an ISP, network administrator, or attacker, can still observe exactly which websites are visited.

This shows a critical vulnerability: **DNS queries leak** browsing habits, even when the content of the communication is secure. Modern mitigations, such as **DNS over HTTPS (DoH)** and **DNS over TLS (DoT)**, aim to encrypt DNS traffic(Herzberg, 2020), but in many networks, standard plaintext DNS is still used(Herzberg, 2020).

ARP Test

To continue our experiment, we again need to facilitate filtering. This time we are going to use **ARP**.



No.	Time	Source	Destination	Protocol	Length	Info
717	208.796977406	0e:69:fd:ac:00:4e	Broadcast	ARP	56	Gratuitous ARP for 10.10.5.145 (Request)
718	208.796978873	0e:69:fd:ac:00:4e	Broadcast	ARP	56	Gratuitous ARP for 10.10.5.145 (Request)
719	209.001324206	e6:83:46:2a:31:28	Broadcast	ARP	56	ARP Announcement for 10.10.4.177
728	212.073571629	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
753	220.572883104	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
795	225.693391926	CeLink_0b:b8:61	Broadcast	ARP	60	ARP Announcement for 10.10.0.93
800	226.409897191	76:c3:04:77:45:b2	Broadcast	ARP	56	ARP Announcement for 10.10.6.55
829	227.740987493	CeLink_0b:b8:61	Broadcast	ARP	60	ARP Announcement for 10.10.0.93
830	228.150347064	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
838	235.830614197	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
845	243.306099586	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
849	248.323504462	0a:e4:f9:db:9d:bd	Broadcast	ARP	56	ARP Announcement for 10.10.4.44
862	248.837202483	Apple_cd:b3:7e	Broadcast	ARP	60	ARP Announcement for 10.10.5.70
883	250.986298549	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
887	251.600457081	82:a4:e2:db:25:03	Broadcast	ARP	64	Gratuitous ARP for 10.10.4.174 (Request)
890	252.624616596	CeLink_0b:b8:61	Broadcast	ARP	60	ARP Announcement for 10.10.0.93
900	254.570317378	82:a4:e2:db:25:03	Broadcast	ARP	56	ARP Announcement for 10.10.4.174
901	254.672769294	CeLink_0b:b8:61	Broadcast	ARP	60	ARP Announcement for 10.10.0.93
902	254.672770691	Apple_cd:b3:7e	Broadcast	ARP	60	ARP Announcement for 10.10.5.70
908	259.075824150	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
915	264.300706055	ExtremeNetwo_8f:c9:...	Intel_6a:c5:0c	ARP	60	Who has 10.10.5.229? Tell 10.10.4.3
916	264.300742722	Intel_6a:c5:0c	ExtremeNetwo_8f:c9:...	ARP	42	10.10.5.229 is at 28:7f:cf:6a:c5:0c
918	271.159325593	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
940	275.869813392	ce:24:aa:d0:13:d1	Broadcast	ARP	64	Gratuitous ARP for 10.10.4.236 (Request)
957	278.841302638	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
965	280.171058396	CeLink_0b:b8:61	Broadcast	ARP	60	ARP Announcement for 10.10.0.93
971	282.116216172	CeLink_0b:b8:61	Broadcast	ARP	60	ARP Announcement for 10.10.0.93
976	286.928732802	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
977	290.001381212	82:a4:e2:db:25:03	Broadcast	ARP	56	Gratuitous ARP for 10.10.4.174 (Request)
978	290.001382608	82:a4:e2:db:25:03	Broadcast	ARP	56	Gratuitous ARP for 10.10.4.174 (Request)
979	290.001384005	82:a4:e2:db:25:03	Broadcast	ARP	56	Gratuitous ARP for 10.10.4.174 (Request)
980	290.103666383	82:a4:e2:db:25:03	Broadcast	ARP	64	Gratuitous ARP for 10.10.4.174 (Request)
981	291.026349504	12:95:c0:0b:95:b3	Broadcast	ARP	56	Gratuitous ARP for 10.10.5.160 (Request)
982	291.026350482	12:95:c0:0b:95:b3	Broadcast	ARP	56	Gratuitous ARP for 10.10.5.160 (Request)
983	291.026351878	12:95:c0:0b:95:b3	Broadcast	ARP	56	Gratuitous ARP for 10.10.5.160 (Request)
989	293.072771995	82:a4:e2:db:25:03	Broadcast	ARP	56	ARP Announcement for 10.10.4.174

(Figure 4.1 - Wireshark with only ARP filter)

Figure 4.1 demonstrates the screen with only **ARP** filtered. Here we need to look for lines like:

- Who has 192.168.1.1? Tell 192.168.1.5

- 192.168.1.1 is at xx:xx:xx:xx:xx:xx

No.	Time	Source	Destination	Protocol	Length	Info
717	208.796977406	0e:69:fd:ac:00:4e	Broadcast	ARP	56	Gratuitous ARP for 10.10.5.145 (Request)
718	208.796978873	0e:69:fd:ac:00:4e	Broadcast	ARP	56	Gratuitous ARP for 10.10.5.145 (Request)
719	209.001324206	e6:83:46:2a:31:28	Broadcast	ARP	56	ARP Announcement for 10.10.4.177
728	212.073571629	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
753	220.572883104	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
795	225.693391926	CeLink_0b:b8:61	Broadcast	ARP	60	ARP Announcement for 10.10.0.93
800	226.409897191	76:c3:04:77:45:b2	Broadcast	ARP	56	ARP Announcement for 10.10.6.55
829	227.740987493	CeLink_0b:b8:61	Broadcast	ARP	60	ARP Announcement for 10.10.0.93
830	228.150347064	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
838	235.830614197	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
845	243.306099586	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
849	248.323504462	0a:e4:f9:db:9d:bd	Broadcast	ARP	56	ARP Announcement for 10.10.4.44
862	248.837202483	Apple_cd:b3:7e	Broadcast	ARP	60	ARP Announcement for 10.10.5.70
883	250.986298549	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
887	251.600457081	82:a4:e2:db:25:03	Broadcast	ARP	64	Gratuitous ARP for 10.10.4.174 (Request)
890	252.624616596	CeLink_0b:b8:61	Broadcast	ARP	60	ARP Announcement for 10.10.0.93
900	254.570317378	82:a4:e2:db:25:03	Broadcast	ARP	56	ARP Announcement for 10.10.4.174
901	254.672769294	CeLink_0b:b8:61	Broadcast	ARP	60	ARP Announcement for 10.10.0.93
902	254.672770691	Apple_cd:b3:7e	Broadcast	ARP	60	ARP Announcement for 10.10.5.70
908	259.075824150	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
915	264.300706055	ExtremeNetwo_8f:c9:f4	Intel_6a:c5:0c	ARP	60	Who has 10.10.5.229? Tell 10.10.4.3
916	264.300742722	Intel_6a:c5:0c	ExtremeNetwo_8f:c9:f4	ARP	42	10.10.5.229 is at 28:7f:cf:6a:c5:0c
918	271.159325593	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
940	275.869813392	ce:24:aa:d0:13:d1	Broadcast	ARP	64	Gratuitous ARP for 10.10.4.236 (Request)
957	278.841302638	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
965	280.171058396	CeLink_0b:b8:61	Broadcast	ARP	60	ARP Announcement for 10.10.0.93
971	282.116216172	CeLink_0b:b8:61	Broadcast	ARP	60	ARP Announcement for 10.10.0.93
976	286.928732802	52:d0:aa:ab:33:e8	Broadcast	ARP	56	ARP Announcement for 10.10.6.16
977	290.001381212	82:a4:e2:db:25:03	Broadcast	ARP	56	Gratuitous ARP for 10.10.4.174 (Request)
978	290.001382608	82:a4:e2:db:25:03	Broadcast	ARP	56	Gratuitous ARP for 10.10.4.174 (Request)
979	290.001384005	82:a4:e2:db:25:03	Broadcast	ARP	56	Gratuitous ARP for 10.10.4.174 (Request)
980	290.103666383	82:a4:e2:db:25:03	Broadcast	ARP	64	Gratuitous ARP for 10.10.4.174 (Request)
981	291.026349504	12:95:c0:0b:95:b3	Broadcast	ARP	56	Gratuitous ARP for 10.10.5.160 (Request)
982	291.026350482	12:95:c0:0b:95:b3	Broadcast	ARP	56	Gratuitous ARP for 10.10.5.160 (Request)
983	291.026351878	12:95:c0:0b:95:b3	Broadcast	ARP	56	Gratuitous ARP for 10.10.5.160 (Request)
989	293.072771995	82:a4:e2:db:25:03	Broadcast	ARP	56	ARP Announcement for 10.10.4.174

(Figure 4.2 - Demonstrating the packets that have to be investigated)

The black-marked packets are the ones we need to continue investigating.

Packets **915** and **916** represent a fundamental exchange on any local network: the Address Resolution Protocol (**ARP**) in action(TechTarget, 2025).

Packet 915: The ARP Request

Source: ExtremeNetwo_8f:c9:f4 (IP: 10.10.4.3)

Destination: Broadcast (This is sent to every device on the local network segment)

Info: Who has 10.10.5.229? Tell 10.10.4.3

This packet is an announcement. The device at 10.10.4.3 should send data to 10.10.5.229, but it doesn't know the physical (MAC) address required to send it at the hardware level. Instead of knowing where to send the request, it screams the question to everyone on the network: "Who owns this IP address? When you find out, tell me directly at my IP address."

Packet 916: The ARP Reply

Source: Intel_6a:c5:0c (IP: 10.10.5.229)

Destination: ExtremeNetwo_8f:c9:f4 (This is sent directly to the requester)

Info: 10.10.5.229 is at 28:7f:cf:6a:c5:0c

The device with the IP address 10.10.5.229 listens for broadcasts and responds directly to the requester. It says, "I am the device you are looking for, and my MAC address is 28:7f:cf:6a:c5:0c." Device 10.10.4.3 receives this answer and adds the IP-MAC pairing to its ARP cache, ensuring it can correctly address future communication.

Why This Exchange is Necessary:

This process is the basis of local network communication. IP addresses are a logical concept for routing between networks, but real data delivery on a local Ethernet or Wi-Fi network depends on MAC addresses. ARP is the essential "translator" that bridges the logical network layer (IP) with the physical network layer (MAC) (TechTarget, 2025; Kurose & Ross, 2021)..

The Critical Vulnerability:

The big security issue with this process is that it doesn't verify that the device sending an ARP reply is actually who it claims to be (Khan, Siddiqui, & Malik, 2023). The system just trusts what it receives. This weakness allows someone malicious on the same network to launch an attack called ARP Spoofing (or ARP Poisoning) (Khan, Siddiqui, & Malik, 2023). Here's how they do it:

- The attacker listens for the ARP Request (Packet 915).
- Then, before the real device answers, the attacker sends a fake ARP Reply saying: "10.10.5.229 is at [Attacker's MAC Address]."
- The device 10.10.4.3 believes this lie and changes its ARP cache to use the attacker's address instead.
- Now, all data intended for 10.10.5.229 is sent to the attacker's machine.
- The attacker sits in the middle (Man-in-the-Middle) and can see, steal, or change all the communication between 10.10.4.3 and 10.10.5.229 without anyone noticing.
- This way, the attacker can quietly spy on or interfere with the data flowing between devices on the network. (Khan, Siddiqui, & Malik, 2023)

Analysis

Three penetration tests were made which are HTTP Login, DNS Lookup, and ARP Spoofing. Together they show a variety of vulnerabilities that typical home routers have.

The tests were done in such a way that they focus on different parts of the network, which revealed ways in which users can be exposed and how routers fail to protect them.

The HTTP Login Test exposed complete credential information in cleartext. Since HTTP traffic is transmitted unencrypted over port 80, the attacker can intercept login data using only passive listening. This shows a critical application-layer vulnerability, due to the router's inability to enforce HTTPS-only browsing or block insecure pages. This finding directly supports the research question by proving that routers in default mode are blind to encrypted traffic risks, a failure confirmed in studies by Anderson (2020) and Chai et al. (2022).

In the DNS Lookup Test, the router failed to hide or encrypt DNS queries. Although the webpage content was encrypted, the domain names visited were fully visible to passive observers. This metadata exposure allows profiling, surveillance, or phishing targeting without breaking HTTPS. Compared to HTTP sniffing, this leak is subtler but still dangerous. The absence of DNS-over-HTTPS or DNS-over-TLS features by default validates the claim that ISPs and router vendors often neglect privacy (Herzberg, 2020).

The ARP Test revealed perhaps the most dangerous vulnerability: a complete lack of verification at the data link layer. Because ARP accepts any response without validation, a malicious actor on the same network could easily perform ARP spoofing. Even though

this test did not include an active spoofing simulation, the conditions were shown to be ideal for such attacks. The router provided no detection, mitigation, or warning.

These results do not occur at the same time; they reinforce each other's impact. HTTP and DNS tests show that data can be seen, while ARP reveals how an attacker could control traffic. Together, they point to a system-wide problem, not just single issues.

Although the three tests that were done, they focused on different OSI layers, and a deeper analysis tells that all the vulnerabilities arise from the choices appearing in underlying protocols. The HTTP protocol relies on TCP and without any cryptography, which is the reason why the payload is transmitted in cleartext as it was constructed in the browser. From computer science, this shows a weakness in the protocols and not in the router. DNS uses UDP and has no authentication or guarantee that the information is confidential, which becomes an explanation why all the information is visible until DNS over HTTPS or DNS over TLS is used. ARP is even more fundamental, it accepts any reply even without authentication, so the ARP cache can be poisoned by any user. So the results reveal that the weaknesses that block router to secure users comes from not router itself, it is linked to historical designs of protocols, where were prioritized simplicity and the speed and not security.

This analysis has some limitations since it focused on just one router model and used only passive testing. Future studies could use active ARP spoofing, compare various router models with threat detection features, or test the impact of DNS encryption for clearer results.

This investigation shows that most basic home routers do not have key security features enabled by default. The essay uses different types of observations, evaluates the risks, and refers to cybersecurity research to answer the research question thoroughly.

Conclusion

This investigation showed that home routers with default settings have serious security problems. Through three tests, it became clear that private information can be captured with little effort. Passwords were sent unencrypted, visited websites were revealed, and local devices could be tricked into sending data to the wrong destination. All of this was possible using just Wireshark on a home network, without any advanced hacking tools.

These findings clearly answer the research question: Home routers are vulnerable to significant level. They do not mitigate or block common, basic threats, and many users are unaware of these risks. This emphasizes why basic network security is important, even at home. A few steps to be more secure on the internet are to turn on HTTPS-only settings, use firewall protection, and consistently update the router's firmware.

Evaluation & Limitations

This investigation showed success revealing key vulnerabilities, but at the same time it had limitations which did not allow to fully uncover the topic and examine the title. One of the main issues that occurred during the work was the restrictions set by internet providers. I tested the routers on two different Internet Provider networks, but both providers had a policy that prevented consumers from accessing admin settings. The result is that I could not enable or disable specific features such as encrypted DNS, HTTPS redirection, or ARP filtering. This has become a limit which led to my inability to compare network behavior with and without security protections, which could have strengthened the results and made them more complete.

Additionally, the tests were conducted on only one router model, which may not be fully representative of all routers, as different models have different default settings. Also, due to time and equipment constraints, the test focused solely on passive capture using Wireshark. More advanced attacks, such as spoofing simulation or real-time packet injection, were not performed, although they would reveal more severe vulnerabilities in detail, leading to a better understanding of the topic and the importance of vulnerabilities.

The decision to rely only on passive testing also influenced how I understand home network security. First I thought the HTTP login test as from my perspective it is unacceptable that it exposed full credentials in cleartext. On the other hand the ARP traffic could be intercepted which did not require any verification, this was a turning point for my focus on low level protocol designs and not only application layer configuration. Here I understood that many vulnerabilities that need to be fixed are not coming from simple user settings, because all the weaknesses and skipped security parts that were

exposed during this investigation are coming from legacy protocols such as ARP and DNS and how they were made originally. If I needed to repeat this investigation I would've combined the passive captures with active spoofing to absorb the speed of detecting or failure of detections of manipulation.

The future work must include active testing and should have ARP spoofing tests, DNS poisoning, and HTTP downgrade trials. These will let us see if the router actively detects and/or blocks the threats. This eventually would change the conclusion part. If certain attacks are mitigated, it would reveal limitations in this passive test approach and give more detailed understanding of home router security.

Bibliography

Anderson, Ross. *Security Engineering: A Guide to Building Dependable Distributed Systems*.

3rd ed., Wiley, 2020.

Auvik Networks. "20 Most Common Network Protocols and Their Port Numbers." *Auvik*,

Auvik Networks Inc.,

<https://www.auvik.com/franklyit/blog/common-network-protocols/>.

Accessed 21 Oct. 2025.

Chai, Huan, Li Zhou, and Yan Zhang. "Security Analysis of Consumer-Grade Routers." *Journal of Network Security*, vol. 15, no. 3, 2022, pp. 45–59.

Experian. "How Many People Were Affected by Data Breaches in 2024?" *Experian News Blog*, 2024, <https://www.experian.com/blogs/news/>. Accessed 21 Oct. 2025.

Herzberg, Amir. "DNS Privacy and Security: Threats and Defenses." *Computer Networks*, vol. 170, 2020, article 107095. <https://doi.org/10.1016/j.comnet.2019.107095>.

Khan, Raza, Ahsan Siddiqui, and Sami Malik. "ARP Spoofing and Mitigation Techniques: A Survey." *International Journal of Cybersecurity Research*, vol. 9, no. 2, 2023, pp. 102–117.

Kurose, James F., and Keith W. Ross. *Computer Networking: A Top-Down Approach*. 8th ed., Pearson, 2021.

TechTarget. "Common Network Protocols." *TechTarget Networking*, TechTarget, <https://searchnetworking.techtarget.com>. Accessed 21 Oct. 2025.

Service Name and Transport Protocol Port Number Registry.
www.iana.org/assignments/service-names-port-numbers.

Wireshark Foundation. "Wireshark User Guide." *Wireshark*,
<https://www.wireshark.org/docs/>. Accessed 21 Oct. 2025.